

Progetto finale del corso di Programmazione per la Fisica: Simulazione dell'interazione fra due specie

Libero Pollini
Alma Mater Studiorum Università di Bologna

25 agosto 2025 (a.a. 2024/25)

Abstract

Utilizzando appositi strumenti software di programmazione in C++, è stato realizzato un semplice programma che simula l'andamento nel tempo del numero di animali di due specie che interagiscono fra loro rispettivamente come prede e predatori. Il modello matematico usato per descrivere il sistema è quello delle due equazioni differenziali di Lotka-Volterra (1926). Il programma, a partire da parametri iniziali forniti dall'utente, calcola punto per punto il valore delle funzioni soluzioni di queste equazioni in un certo intervallo temporale, usando la cosiddetta approssimazione di Eulero.

Indice

1	Link esterni	2
2	Principali scelte implementative	2
3	Indicazioni per l'esecuzione	2
3.1	Esempi (input/output)	3
4	Logica dei test implementati internamente	3
5	Conclusioni	4

1 Link esterni

- Le indicazioni complete per lo svolgimento del progetto sono disponibili alla pagina web: <https://github.com/Programmazione-per-la-Fisica/progetto2024/blob/main/volterra.md>.
- Il progetto completo realizzato, invece, alla pagina: <https://github.com/LiberoP/Final>.
- Lo script in *python* usato per validare i risultati alla pagina: <https://colab.research.google.com/drive/17XdXNRFyBkJBd8jAHqFlpUKDrv18FmVI?usp=sharing>.

2 Principali scelte implementative

L'implementazione del programma segue da vicino le indicazioni generali fornite nelle lezioni del corso e al link indicato.

Il programma è stato suddiviso nei seguenti *header files*: `main.cpp` (principale, implementato secondo le convenzioni standard, gestisce l'input/output; vedi oltre), `final.test.cpp` (dove sono implementati i test con *Doctest*; vedi oltre), `final.hpp` (dove vengono definite classi e struct e dichiarate le funzioni membro), `final.cpp` (dove vengono definite le funzioni precedentemente dichiarate).

Negli ultimi due file, vengono definite le struct `UserPoint` per il punto iniziale fornito dall'utente (cioè di coordinate x_0, y_0), `InternalPoint` per i punti usati nel calcolo ma non mostrati alla fine (cioè di coordinate "relative" $x_{rel,i}, y_{rel,i}, H(x_{rel,i}, y_{rel,i})$), `OutputPoint` per i punti mostrati in output (cioè di coordinate "assolute" $x_i, y_i, H(x_i, y_i)$) e `Parameters` per tutti i parametri forniti dall'utente. Viene poi definita la classe `Simulation`, all'interno della quale vengono dichiarate privatamente come variabili membro il vettore dei punti "interni" `points_`, i parametri forniti dall'utente `pars_` e il punto iniziale `p0_`.

Nella parte pubblica di questa classe vengono implementate le funzioni membro: `size()` che restituisce la dimensione di `points_` e sarà usata in seguito; `addParameters(Parameters const&)` che, dopo aver controllato che i parametri forniti dall'utente abbiano valori validi, cambia la dimensione del vettore `points_` secondo il parametro N ; `addUserPoint(UserPoint const&)` che, dopo aver controllato che siano stati effettivamente aggiunti i parametri e che il punto iniziale abbia coordinate valide, aggiunge il punto al primo posto del vettore, riscalandolo come `InternalPoint`; `evolve()` che, dopo aver controllato che il punto iniziale sia stato effettivamente aggiunto, tramite un ciclo `for` riempie il vettore con i punti di coordinate relative successivi, calcolando cioè l'evoluzione temporale del sistema; `result()` che crea un vettore `result` della stessa dimensione di `points_` riempiendolo con gli stessi punti riconvertiti in coordinate assolute, cioè di tipo `OutputPoint`.

Infine, è implementata la funzione libera `simulation(Simulation const&)`, per poter avere il risultato fuori dalla classe.

3 Indicazioni per l'esecuzione

Il programma può essere facilmente eseguito e testato da terminale con i comandi standard di *CMake* visti a lezione. Tutti i test già implementati nel file `final.test.cpp` vengono superati e non compaiono messaggi d'errore, sia nella modalità *Debug* che nella modalità *Release*.

Inoltre, se compilato ed eseguito direttamente da terminale WSL (comandi `g++ -Wall -Wextra main.cpp -o main final.cpp final.hpp` seguito da `./main`), il programma legge i parametri di input da un file di testo denominato `Input.txt` nella directory di lavoro e scrive i risultati in un file denominato `Results.txt` nella stessa cartella. In questo secondo caso, l'utente dovrà quindi creare innanzitutto un file di input con quel nome e modificarlo (comandi `touch Input.txt` seguito da `wslview Input.txt` e modifica del file), inserendo i valori dei parametri separati solo da spazi in quest'ordine: $A, B, C, D, N, \Delta t, x_0, y_0$. L'utente potrà poi leggere i risultati in forma tabulare stampando a schermo il contenuto del file di output (comando `cat Results.txt`).

3.1 Esempi (input/output)

Il corretto funzionamento del programma si può verificare direttamente da terminale, modificando il file di input, poi compilando ed eseguendo e leggendo il contenuto del file di output. Ad esempio:

1. Si riceve un messaggio d'errore esplicativo al momento dell'esecuzione in ognuno dei seguenti casi:
 - lasciando il file di input vuoto;
 - non fornendo tutti i parametri iniziali;
 - fornendoli in modo errato (ad esempio separati da virgole);
 - fornendoli in un formato sbagliato (non tutti numeri reali positivi, oppure N non intero).
2. Fornendo in input i parametri 1.2 0.5 0.1 0.5 100 0.001 20. 10. (test "final C1") si riceve in output una tabella di 100 righe la cui ultima riga è 0.1 4.07738 20.3308 6.25584
3. Fornendo in input i parametri 1.2 0.5 0.1 0.5 100 0.001 10. 20. (test "final C2") si riceve in output una tabella di 100 righe la cui ultima riga è 0.1 4.05678 20.5333 6.48572
4. Fornendo in input i parametri 1.2 0.5 0.1 0.5 100 0.001 10. 10. (test "final C3") si riceve in output una tabella di 100 righe la cui ultima riga è 0.1 6.76782 10.3362 2.08599
5. Fornendo in input i parametri 1.3 0.4 0.1 0.5 100 0.001 171. 34. (test "final D4") si riceve in output una tabella di 100 righe la cui ultima riga è 0.1 19.2652 71.6715 23.5622
6. Fornendo in input i parametri 1.3 0.4 0.1 0.5 100 0.001 18. 299. (test "final D5") si riceve in output una tabella di 100 righe la cui ultima riga è 0.1 7.08698e-05 288.76 112.916
7. Fornendo in input i parametri 1.3 0.4 0.1 0.5 100 0.001 231. 227. (test "final D6") si riceve in output una tabella di 100 righe la cui ultima riga è 0.1 0.00254422 271.855 104.442

4 Logica dei test implementati internamente

Oltre a controllare direttamente da terminale che il programma funzionasse correttamente, sono stati implementati con *Doctest* diversi test (non riguardanti la parte di input/output) nel file `final.test.cpp` del programma stesso e si è verificato con *CMake* che fossero soddisfatti dal programma implementato. Questi test controllano che:

1. Il programma restituisca un messaggio d'errore se i parametri A , B , C , D dell'equazione iniziale e N , Δt per il calcolo approssimato non sono forniti;
2. Il programma restituisca un messaggio d'errore se non tutti questi parametri hanno valori reali strettamente positivi, o se N non è un intero;
3. Il programma restituisca un messaggio d'errore se le coordinate del punto iniziale $x_0 \equiv x(t = 0)$ e $y_0 \equiv y(t = 0)$ non sono fornite;
4. Il programma restituisca un messaggio d'errore se queste coordinate non hanno valore reale strettamente positivo;
5. A partire da diversi (12 in totale) input di parametri iniziali, i valori delle funzioni $x(t)$, $y(t)$ e $H(x, y)$ a certi step (quello finale e uno intermedio) calcolati dal programma in modo approssimato siano compatibili, entro un'incertezza del 5%, con quelli "veri" calcolati con l'apposito script in *python*. L'elevata accuratezza di questi ultimi è garantita dal fatto che l'integrale primo del moto H rimane quasi esattamente costante nel tempo nei casi considerati dai test. Inoltre in pochi casi (segnalati all'interno del file di test), per valori di $x(t)$ e $y(t)$ nell'ordine delle unità o inferiori, si è dovuto aumentare l'incertezza tollerata nei test fino a un massimo del 25%.

5 Conclusioni

Nel complesso, il programma implementato realizza tutti gli obiettivi basilari richiesti dalla consegna. Visto il limitato tempo a disposizione, si è posta particolare attenzione alla gestione delle *exceptions*, alla correttezza formale e al rispetto delle convenzioni del C++, a scapito della realizzazione di funzionalità più complesse.